

Principal-Realtime-Assessment – Candidate Brief

Overview

Design and implement a real-time backend that ingests IoT telemetry, processes it, and pushes results to a WebSocket UI. This is open-book and unproctored. We evaluate architecture, correctness, performance trade-offs, and clear reasoning.

Scenario

≈2,000 devices send JSON events. Sustained ingress: ≈3,000 msgs/min (≈50/s) with bursts up to 10× for 2 minutes. Each message should be processed and pushed ASAP to the UI.

Service Level Objectives (SLOs)

- End-to-end latency (ingest → process → push): $p_{95} \leq 800$ ms; $p_{99} \leq 1200$ ms at 3k/min.
- No message loss. Preserve per-device ordering on writes. Input may contain duplicates (at-least-once).

Input Message (example)

```
{ "deviceId": "str", "ts": "RFC3339", "type":  
  "temp|hum|vibration", "value": number, "seq": long }
```

Processing requirements

- Broadcast processed events on the WebSocket channel to all connected clients.

Tech constraints (choose & justify)

- Java 17+, Spring Boot 3, WebFlux, Spring Security.
- Kafka or MQTT for ingest (your choice; justify).
- PostgreSQL (TimescaleDB optional). Use SQL migrations.
- WebSocket for UI updates. Containerized runnable solution (Docker Compose minimum; K8s optional).

What to deliver

- Architecture doc (≤ 6 pages): dataflow; partitioning & keys; backpressure; idempotency & ordering; DB schema/indexes; auth; observability; SLOs & risks.

- Working vertical slice: ingest → process → persist → broadcast. Include scripts/commands to run end-to-end.
- Load/perf report: methodology; sustained vs burst behavior; p50/p95/p99 latencies; throughput; CPU; heap/GC (graphs or tables).
- Tests: meaningful unit tests and integration tests (prefer Testcontainers).
- Security proof: enable a secure profile with JWT validation (and outline mTLS plan between services).
- Live demo with recruiter: present your solution and walk through the architecture and a short load demo.
- Code review: written review of the provided review-exercise module with concrete fixes and rationale.

How we evaluate

- Architectural choices & trade-offs (explicit reasoning).
- Correctness under at-least-once delivery and per-device ordering.
- Latency/backpressure behavior at steady state and during bursts.
- Code quality, tests, and observability (metrics/logs/traces).
- Security posture (JWT in secure profile; plan for mTLS/secrets).
- Clarity of write-ups and the PR review.

Environment & how to run

Start infrastructure (Docker Compose):

```
$ cp .env.example .env
$ docker compose up -d
```

Bootstrap Gradle wrapper and run backend:

```
$ cd backend
$ ./bootstrap-wrapper.sh      # macOS/Linux
$ bootstrap-wrapper.bat      # Windows
$ ./gradlew clean bootRun
$ curl -i http://localhost:8080/actuator/health
```

WebSocket test UI:

Open ui/index.html in your browser and connect to ws://localhost:8080/ws.

Kafka generator (on host to external listener):

```
$ cd generator
$ npm install
$ node kafka-producer.js --broker=localhost:29092 --topic=telemetry
  --rate=50 --burst=500 --burstSeconds=120
```

MQTT generator:

```
$ cd generator
$ npm install
$ node mqtt-producer.js --url=mqtt://localhost:1883
  --topic=telemetry --rate=50 --burst=500 --burstSeconds=120
```

Optional: k6 WebSocket smoke test:

```
$ cd load
dockerrun --rm --i --networkhost --vPWD:/scripts" grafana/k6
  run /scripts/k6.js
```